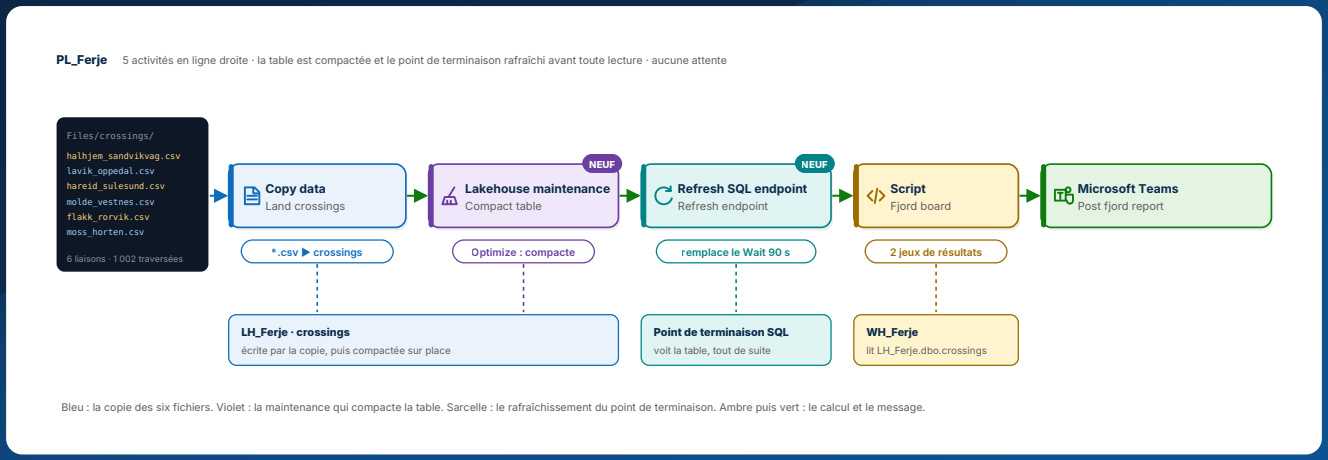
MICROSOFT FABRIC · DATA FACTORY · LAKEHOUSE MAINTENANCE · GUIDE PAS À PAS

Pipeline Microsoft Fabric : il range derrière lui

CHARGER COMPACTER RAFRAÎCHIR MESURER PRÉVENIR

Norvège, octobre 2026. Six liaisons de ferries électriques, six fichiers, une copie. Je vous montre comment le pipeline fait ensuite le ménage lui-même : il compacte la table et rafraîchit le point de terminaison SQL. Zéro attente.



5	2	1 002	2,2×	0 s
activités	activités neuves	traversées	d'écart d'énergie	d'attente

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

FICHIERS DU PROJET
6 CSV dans crossings, fjord_board.sql, controle.sql, expressions.txt

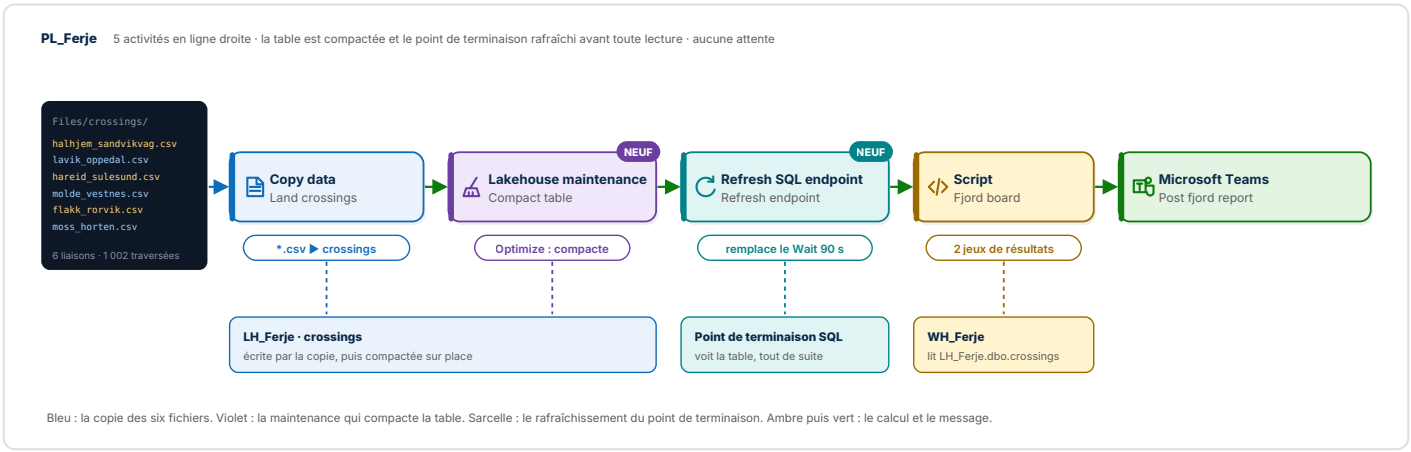
PRÉREQUIS
Un espace de travail Fabric sur une capacité, Teams avec votre compte

DURÉE
Environ 25 minutes, en quatre étapes vérifiées une à une

VUE D'ENSEMBLE

Ce que nous allons construire

Chaque lundi, les six liaisons envoient leur semaine : un CSV par liaison, une ligne par traversée (passagers, voitures, kWh consommés, batterie restante). `PL_Ferje` charge les six fichiers en une copie, puis **range derrière lui** : la **maintenance du lakehouse** compacte la table, et le **rafraîchissement du point de terminaison SQL** rend les nouvelles lignes visibles tout de suite. Le Script lit donc une table propre et à jour, sans la pause de 90 secondes que l'on ajoute habituellement. Ce que vous allez apprendre : **l'activité Lakehouse maintenance**, **l'activité Refresh SQL analytics endpoint**, **une connexion Lakehouse partagée** et **la copie de plusieurs fichiers par un chemin générique**.



AVANT DE COMMENCER : 30 SECONDES DE VÉRIFICATION

Ces deux activités sont récentes. Créez un pipeline vide, ruban **Activités**, et cherchez-les. Les trois autres activités sont classiques.

ACTIVITÉ	CHERCHEZ	SI ELLE MANQUE
Lakehouse maintenance NEUF	`maintenance`	Plan B : un notebook d'une ligne, `OPTIMIZE`
Refresh SQL analytics endpoint NEUF	`Refresh SQL` ou `Actualiser`	Plan B : un Wait de 90 s, comme avant

Les activités

ACTIVITÉ	TYPE	RÔLE
`Land crossings`	Copy data · générique	Lit `*.csv` dans `crossings` et écrit les 1 002 traversées dans la table `crossings`
`Compact table`	Lakehouse maintenance	**Optimize** : regroupe les petits fichiers Parquet de la table en gros fichiers
`Refresh endpoint`	Refresh SQL analytics endpoint	Force le point de terminaison SQL à voir la table maintenant
`Fjord board`	Script	Sur `WH_Ferje` : résultat 0, la semaine ; résultat 1, la liaison la plus sobre et la plus gourmande
`Post fjord report`	Microsoft Teams	Le bilan de la semaine en une ligne

POURQUOI C'EST UN SAUT D'habitude, on ajoute une attente de 90 secondes en espérant que le point de terminaison ait vu la table. Ici, on lui **demande** de se mettre à jour, et on passe à la suite dès qu'il a fini.

TROIS EXÉCUTIONS 1 : la copie seule. 2 : copie, maintenance et rafraîchissement. 3 : le pipeline complet, jusqu'à Teams.

ÉTAPE 1

Le lakehouse, les six fichiers, l'entrepôt

1 · OBJETS | 2 · CHARGER | 3 · RANGER | 4 · MESURER ET PRÉVENIR

1 LH_Ferje — LAKEHOUSE

1. Nouvel espace de travail **Fabric_Pipeline_13**. **Nouvel élément**, **Lakehouse**, nom `LH_Ferje`.
2. Si la case **Schémas de lakehouse** apparaît : **décochez-la**, puis **Créer**.
3. Dans **Fichiers**, **Nouveau sous-dossier** : `crossings`.
4. Dans `crossings`, **Charger**, **Charger des fichiers** : les **six** CSV en une fois.

LA CASE QUI COMPTE La maintenance du lakehouse ne fonctionne pas encore sur un lakehouse **avec schémas**. Sans la case, la table vit simplement dans `Tables`.

2 WH_Ferje — ENTREPÔT

1. **Nouvel élément**, **Entrepôt**, nom `WH_Ferje`. Rien à créer : il lira le lakehouse par son nom à trois parties.

Les premières lignes de Lavik-Oppedal :

```
crossing_id,route,departed_at,minutes,passengers,cars,energy_kwh,battery_end_pct
13147,Lavik-Oppedal,2026-10-12 05:35:00,20,48,48,532,80
13148,Lavik-Oppedal,2026-10-12 06:06:00,20,93,33,486,60
13149,Lavik-Oppedal,2026-10-12 06:47:00,20,114,38,487,73
13150,Lavik-Oppedal,2026-10-12 07:23:00,20,89,41,513,55
13151,Lavik-Oppedal,2026-10-12 07:58:00,20,80,23,449,74
… 202 traversées
```

COLONNE	CONTENU
`crossing_id`	numéro unique de la traversée
`route` · `minutes`	la liaison et sa durée, de 20 à 40 min
`passengers` · `cars`	les personnes et les voitures à bord
`energy_kwh`	l'électricité consommée par la traversée
`battery_end_pct`	la batterie restante à l'arrivée

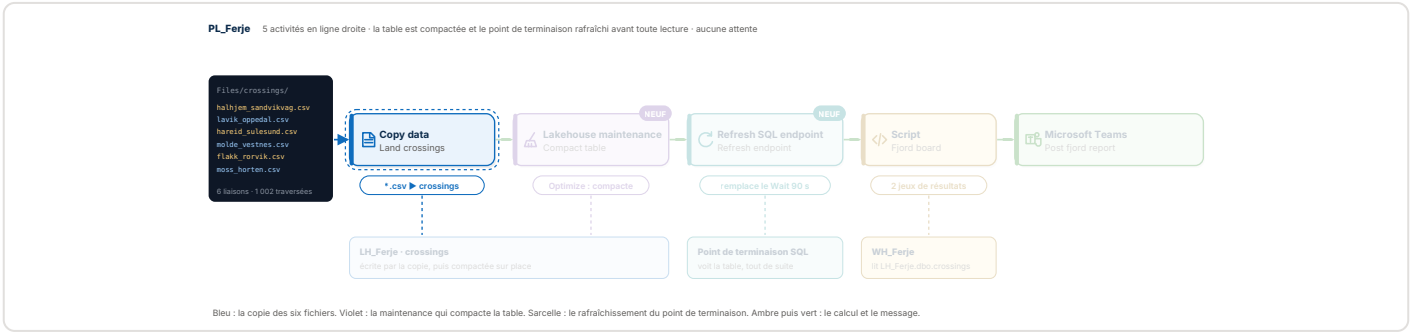
SIX FICHIERS, UNE TABLE Les six CSV ont les mêmes colonnes. Un seul chemin générique, `*.csv`, les lira tous d'un coup.

AVANT LE PIPELINE ☐ `LH_Ferje` sans schémas ☐ 6 CSV dans `crossings` ☐ `WH_Ferje` créé ☐ même espace

ÉTAPE 2

Six fichiers, une copie

1 · OBJETS | 2 · CHARGER | 3 · RANGER | 4 · MESURER ET PRÉVENIR



3 PL_Ferje

PIPELINE

Nouvel élément, **Pipeline**, nom `PL_Ferje`. Pas de paramètre : la semaine entière est dans le dossier.

4 Land crossings

COPY DATA · GÉNÉRIQUE

Source, connexion	`LH_Ferje`
Dossier racine	Fichiers
Type de chemin	Chemin de fichier générique
Dossier	`crossings`
Nom de fichier	`*.csv`
Format	Texte délimité, La première ligne est un en-tête

Destination	`LH_Ferje`, Tables, nouvelle table `crossings`
Action de table	Remplacer

GÉNÉRIQUE : UN MOTIF, PAS UN NOM `*.csv` veut dire « tous les CSV du dossier ». Demain, une septième liaison arrive ? Déposez son fichier, rien à changer dans le pipeline.

REMPLACER, PAS AJOUTER La table est réécrite à chaque exécution : vous pouvez relancer autant de fois que vous voulez, toujours 1 002 lignes.

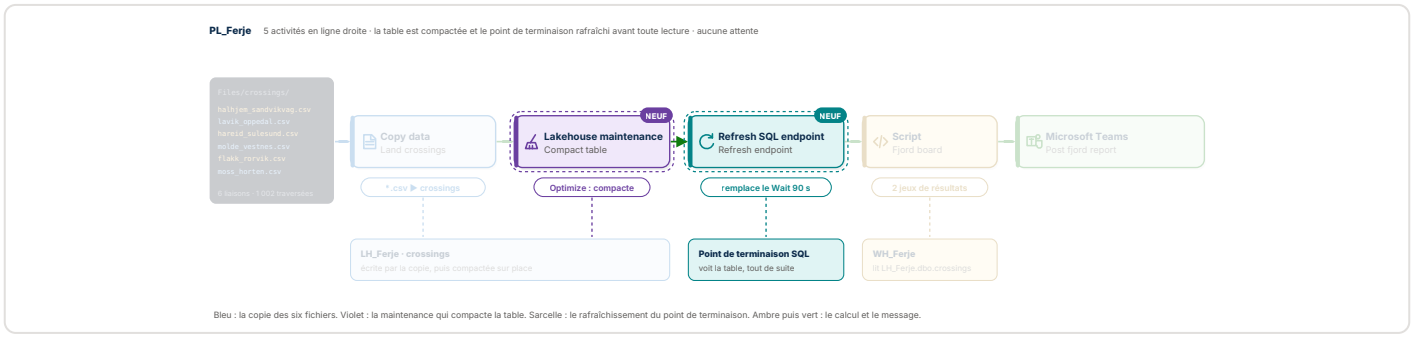
Exécution 1 la copie seule

Enregistrer, **Exécuter**. Sortie de la copie : `filesRead` = **6**, `rowsCopied` = **1 002**. Dans `LH_Ferje`, **Tables** : `crossings`.

ÉTAPE 3

Le pipeline range derrière lui

1 · OBJETS | 2 · CHARGER | **3 · RANGER** | 4 · MESURER ET PRÉVENIR



5 Compact table

LAKEHOUSE MAINTENANCE · NEUF

1. Activité **Lakehouse maintenance**, nom `Compact table`, reliée à `Land crossings` par la flèche verte.
2. Onglet **Paramètres**, **Connexion** : **Parcourir tout**, onglet **Nouveau**, connecteur **Lakehouse**. Nom `CN_Lakehouse`, **Compte d'organisation**, **Se connecter**, **Connecter**.
3. **Lakehouse** : `LH_Ferje`. **Nom de la table** : `crossings`. Laissez **Schéma** vide.
4. **Opération** : **Optimize**.

CE QUE FAIT OPTIMIZE Chaque écriture Delta laisse des fichiers Parquet. Au fil des chargements, ils s'accumulent et ralentissent les lectures. Optimize les fusionne en quelques gros fichiers, sans changer une seule ligne.

UNE VRAIE CONNEXION, PAS LE LAKEHOUSE Choisir directement `LH_Ferje` dans **Connexion** ne suffit pas : la validation répond « artifactDmtsCredential est obligatoire ». Il faut une connexion **Lakehouse** avec votre compte. Elle n'est liée à aucun lakehouse précis : créée une fois, elle sert partout.

6 Refresh endpoint

REFRESH SQL ENDPOINT · NEUF

1. Activité **Refresh SQL analytics endpoint**, nom `Refresh endpoint`, reliée à `Compact table`.
2. **Connexion** : la même, `CN_Lakehouse`.
3. **Espace de travail** : `Fabric_Pipeline_13`. **Point de terminaison analytique SQL** : `LH_Ferje`.

FINI, LE WAIT DE 90 S Le point de terminaison SQL met à jour sa vue des tables en arrière-plan, avec un délai. Cette activité le déclenche tout de suite et attend qu'il ait fini : la suite lit forcément la table à jour.

Exécution 2 copie, maintenance, rafraîchissement

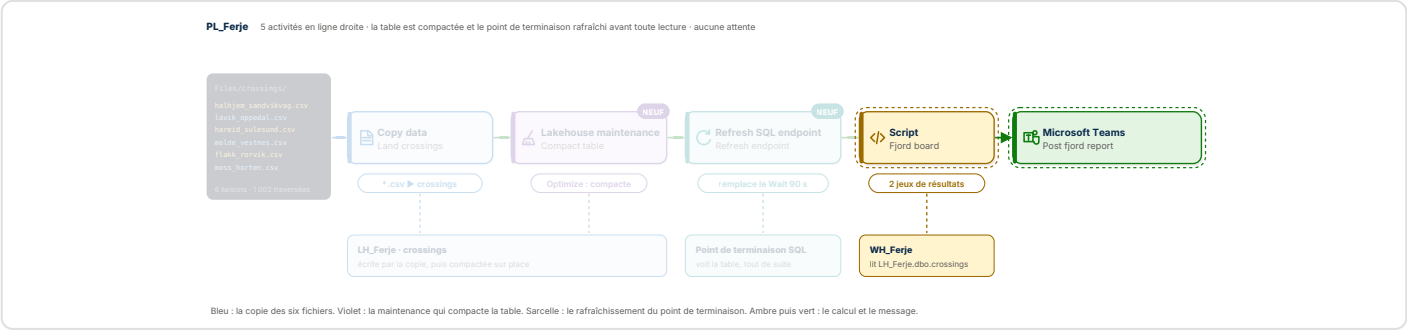
Trois activités vertes. Dans la sortie de `Refresh endpoint` : statut **Success** (ou **NotRun** si le point de terminaison était déjà à jour : c'est normal aussi).

L'ORDRE COMPTE Compacter d'abord, rafraîchir ensuite : le point de terminaison découvre directement la table propre, en peu de fichiers.

ÉTAPE 4

Mesurer l'énergie, prévenir Teams

1 · OBJETS | 2 · CHARGER | 3 · RANGER | **4 · MESURER ET PRÉVENIR**



7 Fjord board — SCRIPT · 2 RÉSULTATS

1. **Script**, nom `Fjord board`, relié à `Refresh endpoint`. Connexion `WH_Ferje`, type **Requête**.
2. Collez les deux requêtes de `fjord_board.sql` (ligne **[Script]**).

```
WITH c AS (SELECT route, CAST(passengers AS int) AS pax, CAST(cars AS int) AS cars,
CAST(energy_kwh AS int) AS kwh, CAST(battery_end_pct AS int) AS batt
  FROM LH_Ferje.dbo.crossings)
SELECT COUNT(*) AS crossings, SUM(pax) AS passengers, SUM(cars) AS cars,
  CAST(SUM(kwh) / 1000.0 AS decimal(8,1)) AS mwh,
  CAST(SUM(kwh) * 1.0 / SUM(cars) AS decimal(6,2)) AS kwh_per_car,
  MIN(batt) AS min_battery FROM c;

WITH c AS (SELECT route, CAST(cars AS int) AS cars, CAST(energy_kwh AS int) AS kwh
  FROM LH_Ferje.dbo.crossings),
r AS (SELECT route, CAST(SUM(kwh) * 1.0 / SUM(cars) AS decimal(6,2)) AS kwh_per_car
FROM c GROUP BY route)
SELECT (SELECT TOP 1 route FROM r ORDER BY kwh_per_car ASC) AS best_route,
  MIN(kwh_per_car) AS best_kwh,
  (SELECT TOP 1 route FROM r ORDER BY kwh_per_car DESC) AS worst_route,
MAX(kwh_per_car) AS worst_kwh FROM r;
```

KWH PAR VOITURE Une liaison longue consomme plus, c'est normal. Ce qui compte : l'énergie divisée par les voitures transportées. On compare l'efficacité, pas la taille.

8 Post fjord report — MICROSOFT TEAMS

1. **Microsoft Teams**, nom `Post fjord report`, relié à `Fjord board`.
2. Titre `Ferries de Norvège`, message : ligne **[Message]**, en **contenu dynamique**.

Exécution 3 — le pipeline complet

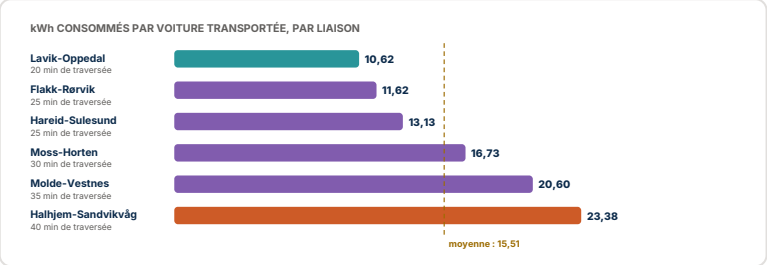
Enregistrer, **Exécuter**. Cinq activités vertes, aucune attente. Dans Teams :



LE TEST QUI PROUVE TOUT Lancez l'exécution 3 juste après avoir rechargé un CSV : le Script trouve tout de suite les 1 002 lignes. Sans `Refresh endpoint`, il aurait pu échouer sur « Invalid object name ».

RÉFÉRENCE

Vérifier, plan B, débloquer



798,6 MWh
consommés en une semaine par six liaisons

2,2×
d'énergie par voiture entre Halhjem-Sandvikvåg et Lavik-Oppedal

123 566
passagers et 51 494 voitures transportés

35 %
la batterie la plus basse à l'arrivée

Ce qu'attend le contrôle

Requête **[Contrôle]** dans `WH_Ferje` : 6 lignes, de la plus sobre à la plus gourmande.

LIAISON	TRAV.	KWH/V.
Lavik-Oppedal	202	**10.62**
Flakk-Rørvik	188	11.62
Hareid-Sulesund	174	13.13
Moss-Horten	132	16.73
Molde-Vestnes	160	20.60
Halhjem-Sandvikvåg	146	23.38

PLAN B · PAS DE LAKEHOUSE MAINTENANCE
Notebook `NB_Ferje`, lakehouse `LH_Ferje`, une cellule : `spark.sql("OPTIMIZE crossings")`. Activité **Notebook** à la place de `Compact table`.

PLAN B · PAS DE REFRESH SQL ENDPOINT Activité **Attendre**, `90` secondes, à la place de `Refresh endpoint`.

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
Copie : « aucun fichier trouvé »	**Type de chemin** doit être **Chemin de fichier générique**, avec `*.csv` en nom de fichier.
Compact table échoue, « schema »	Le lakehouse a été créé avec schémas. Plan B notebook, ou recréez `LH_Ferje` sans la case.
Table introuvable dans Compact table	La copie n'a jamais tourné, ou la table porte un autre nom. Exécution 1 d'abord, nom exact `crossings`.
Refresh endpoint : statut **NotRun**	Pas une erreur : le point de terminaison était déjà à jour. La suite fonctionne.
Script : « Invalid object name »	Le rafraîchissement n'est pas relié avant le Script, ou pointe un autre lakehouse. Vérifiez la flèche et `LH_Ferje`.
Validation : « artifactDmtsCredential est obligatoire »	La connexion pointe l'élément `LH_Ferje`. Sélectionnez `CN_Lakehouse`, puis le lakehouse.
Échec « AADSTS700082 »	Connexion expirée. Recréez-la avec **Compte d'organisation**.